Java

Основы языка

От первой строки до небольшого приложения



Маршрут по языку

- 1. Первая программа, значения и условия
- 2. Циклы, массивы и методы
- 3. Классы, объекты и ссылки
- 4. Интерфейсы и организация кода
- 5. Коллекции, ошибки и небольшое приложение

После каждого блока пишем код самостоятельно.

Как работаем с примерами

- Сначала читаем код и предсказываем результат
- Запускаем и объясняем каждую строку
- Меняем условие, значение или действие
- Пишем похожую программу без копирования

Уметь объяснить программу важнее, чем перепечатать её.

01. Первая программа

Читаем код, сохраняем значения, принимаем решения

Hello World

Полный файл Main.java

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Main.java

Программа выводит
одну строку
в консоль.

Начинаем выполнение
с метода main.

Результат: Hello, world!

Что означает оболочка программы

Фрагмент	Смысл в нашем примере
class Main	Объявляем класс с именем Main
public	Делаем класс или метод доступным извне
static	Для вызова main не создаём объект Main
void	Метод не возвращает значение
String[] args	Параметр с аргументами командной строки
System.out.println(...)	Вызываем метод вывода строки

Класс пока служит местом, где объявлен наш метод.

Сборка и запуск

Команды терминала и результат

```
javac Main.java
java Main

Hello, world!
```

javac компилирует исходный файл.

java запускает класс.

IDE выполняет эти шаги за нас.

Имя public-класса Main совпадает с именем файла Main.java.

Скобки, точки и точка с запятой

Фрагмент внутри main

```
// Комментарий для человека
int count = 3;
System.out.println(count);

if (count > 0) {
    System.out.println("Есть задачи");
}
```

; завершает
эти инструкции.

{ } ограничивают блок.

( ) содержат условие
или аргументы вызова.

Регистр важен: Main и main — разные имена.

Переменная: тип, имя и значение

Фрагмент внутри main

```
int count = 3;
System.out.println(count);

count = 5;
count = count + 1;
System.out.println(count);
```

int — тип.
count — имя.
3 — начальное значение.

= присваивает
новое значение.

Тип count остаётся int при каждом присваивании.

Восемь примитивных типов

Тип	Что хранит	Пример
byte / short	Целое, 8 / 16 бит	byte level = 5;
int	Целое, 32 бита	int count = 10;
long	Целое, 64 бита	long id = 42L;
float / double	Приближённое дробное число	double ratio = 0.5;
boolean	true или false	boolean done = false;
char	Одну кодовую единицу UTF-16	char letter = 'A';

Чаще всего начнём с int, double и boolean.

Компилятор проверяет типы

Фрагмент внутри main

```
int count = 3;
// count = "три";  // ошибка компиляции

double average = count;
int truncated = (int) 3.9;

System.out.println(average);
System.out.println(truncated);
```

Строка не подходит переменной int.

(int) явно сужает тип и отбрасывает дробную часть.

Приведение типа может потерять данные.

Арифметика и целочисленное деление

Фрагмент внутри main

```
System.out.println(7 + 2);
System.out.println(7 / 2);
System.out.println(7 / 2.0);
System.out.println(7 % 2);

int count = 1;
count++;
System.out.println(count);
```

+ - * /
обычная арифметика.

% даёт остаток.

count++ увеличивает значение на один.

7 / 2 равно 3. Для дробного результата нужен дробный операнд.

String: текст и операции с ним

Фрагмент внутри main

```
String title = "  Java  ";
String clean = title.trim();

System.out.println(clean);
System.out.println(clean.length());
System.out.println(clean.toUpperCase());
System.out.println("Тема: " + clean);
```

String — ссылочный тип, не примитив.

Методы вызываются через точку.

trim возвращает другую строку.

Строки неизменяемы: исходный title не изменился.

Что выведет программа?

Фрагмент внутри main

```
int a = 7;
int b = 2;

System.out.println(a / b);
System.out.println("Сумма: " + a + b);
System.out.println("Сумма: " + (a + b));
```

Обсудите в парах.

Запишите три строки вывода.

Потом проверьте свои предположения.

Две минуты без запуска программы.

Разбор: деление и сложение строк

Выражение	Результат	Почему
a / b	3	Оба операнда имеют тип int
"Сумма: " + a + b	Сумма: 72	Слева направо дописываем в строку
"Сумма: " + (a + b)	Сумма: 9	Сначала складываем числа в скобках

Один знак + выполняет разные операции в зависимости от типов.

Сравнения и логические выражения

Фрагмент внутри main

```
int score = 75;
boolean submitted = true;

boolean passed = score >= 60 && submitted;
System.out.println(passed);
System.out.println(score == 100);
System.out.println(!submitted);
```

== равно
!= не равно
< > <= >=

&& логическое И
|| логическое ИЛИ
! отрицание

Условие if должно иметь тип boolean.

if, else if, else

Фрагмент внутри main

```
int score = 75;

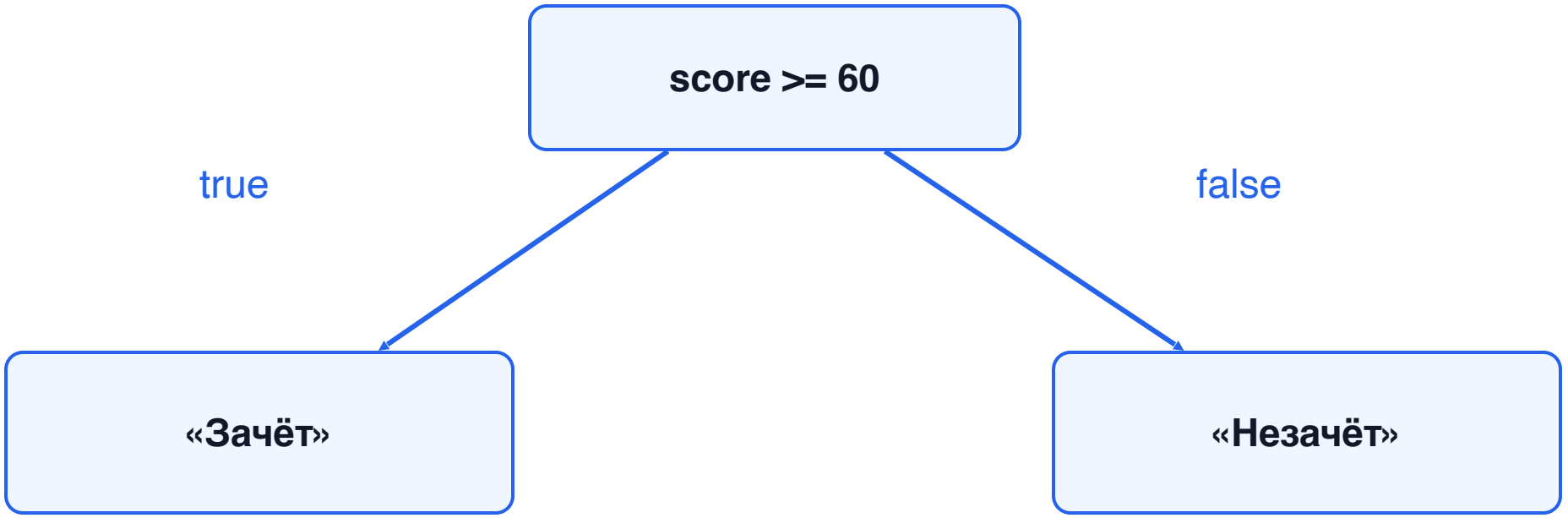
if (score >= 90) {
    System.out.println("Отлично");
} else if (score >= 60) {
    System.out.println("Зачёт");
} else {
    System.out.println("Незачёт");
}
```

Условия проверяются сверху вниз.

Выполняется первая подходящая ветка.

При score = 75 выводится «Зачёт».

Условие выбирает путь



После выбранной ветки выполнение продолжается дальше.

switch: выбор из нескольких вариантов

Фрагмент внутри main

```
int command = 2;

String action = switch (command) {
    case 1 -> "Создать";
    case 2 -> "Показать";
    case 3 -> "Завершить";
    default -> "Неизвестная команда";
};
System.out.println(action);
```

switch здесь возвращает значение.

case задаёт вариант.

default обрабатывает остальные значения.

В форме со стрелками выполнение не проваливается в следующий case.

Читаем ввод пользователя

Фрагмент внутри main

```
Scanner input = new Scanner(System.in);

System.out.println("Как вас зовут?");
String name = input.nextLine();

System.out.println("Привет, " + name);
```

Scanner читает ввод из консоли.

nextLine ждёт строку и возвращает её после Enter.

Над class Main добавляем import java.util.Scanner;

Практика: стоимость доставки

- Есть int total: сумма заказа в целых рублях
- От 2 000 рублей доставка бесплатна
- Иначе доставка стоит 200 рублей
- Выведите сумму заказа вместе с доставкой

Проверьте total = 1999, 2000 и 2500.

Разбор: стоимость доставки

Фрагмент внутри main

```
int total = 1999;
int delivery;

if (total >= 2000) {
    delivery = 0;
} else {
    delivery = 200;
}
System.out.println(total + delivery);
```

1999 → 2199
2000 → 2000
2500 → 2500

В каждой ветке задаём delivery.

Локальную переменную нужно инициализировать до чтения.

02. Повторение и методы

Цикл обрабатывает много значений, метод объединяет действия

while: повторяем, пока условие истинно

Фрагмент внутри main

```
int number = 1;

while (number <= 3) {
    System.out.println(number);
    number++;
}
System.out.println("Готово");
```

1. Проверить условие.
2. Выполнить тело.
3. Вернуться к проверке.

Тело может
не выполниться
ни разу.

Внутри цикла должно происходить то, что приближает его завершение.

for: счётчик в одном месте

Фрагмент внутри main

```
for (int i = 0; i < 3; i++) {
    System.out.println(i);
}
```

int i = 0
начальное значение.

i < 3
условие продолжения.

i++
шаг после тела.

Вывод: 0, 1, 2. Значение 3 в тело не попадёт.

Следим за значениями в цикле

Перед проверкой	i < 3	Действие	После i++
i = 0	true	Вывести 0	i = 1
i = 1	true	Вывести 1	i = 2
i = 2	true	Вывести 2	i = 3
i = 3	false	Выйти из цикла	Шаг не выполняется

Таблица значений помогает найти ошибку без угадывания.

do-while, break и continue

Фрагмент внутри main

```
int n = 0;
do {
    n++;
} while (n < 1);

for (int i = 0; i < 5; i++) {
    if (i == 1) continue;
    if (i == 3) break;
    System.out.println(i);
}
```

do-while проверяет условие после тела.

continue пропускает остаток итерации.

break завершает ближайший цикл.

Цикл for выведет 0 и 2.

Массив: несколько значений одного типа

Фрагмент внутри main

```
int[] scores = {70, 45, 90, 60};

System.out.println(scores[0]);
scores[1] = 55;
System.out.println(scores.length);

int[] emptyScores = new int[3];
System.out.println(emptyScores[0]);
```

int[] — тип массива.

Индексы начинаются с нуля.

length — длина. После создания она не меняется.

new int[3] создаёт три элемента со значением 0.

Индекс и длина массива

```
int[] scores = {70, 45, 90, 60};
```



Длина равна 4. Последний допустимый индекс равен 3.

Перебираем массив

Фрагмент внутри main

```
int[] scores = {70, 45, 90, 60};

for (int score : scores) {
    System.out.println(score);
}

for (int i = 0; i < scores.length; i++) {
    scores[i] = scores[i] + 1;
}
```

for-each удобен,
когда нужен элемент.

Обычный for нужен,
когда нужен индекс
или меняем ячейку.

score — очередное значение. i — индекс элемента.

Практика: сколько студентов сдали?

- Дан массив {70, 45, 90, 60}
- Проходной балл равен 60
- Посчитайте количество оценок не ниже проходной
- Проверьте пустой массив и массив {59, 60}

Сначала опишите алгоритм словами, затем напишите цикл.

Разбор: счётчик подходящих элементов

Фрагмент внутри main

```
int[] scores = {70, 45, 90, 60};
int passed = 0;

for (int score : scores) {
    if (score >= 60) {
        passed++;
    }
}
System.out.println(passed);
```

Счётчик создаём до цикла.

Увеличиваем только при выполнении условия.

Ответ: 3. Для пустого массива ответ 0.

Метод: именованный фрагмент поведения

Метод внутри класса Main, рядом с main

```
static int total(int price, int count) {
    return price * count;
}
```

total — имя метода.

price и count — параметры типа int.

int перед именем — тип результата.

Метод объявляем в теле класса, вне main.

Объявление и вызов метода

Фрагмент внутри main

```
int first = total(100, 3);
int second = total(50, 2);

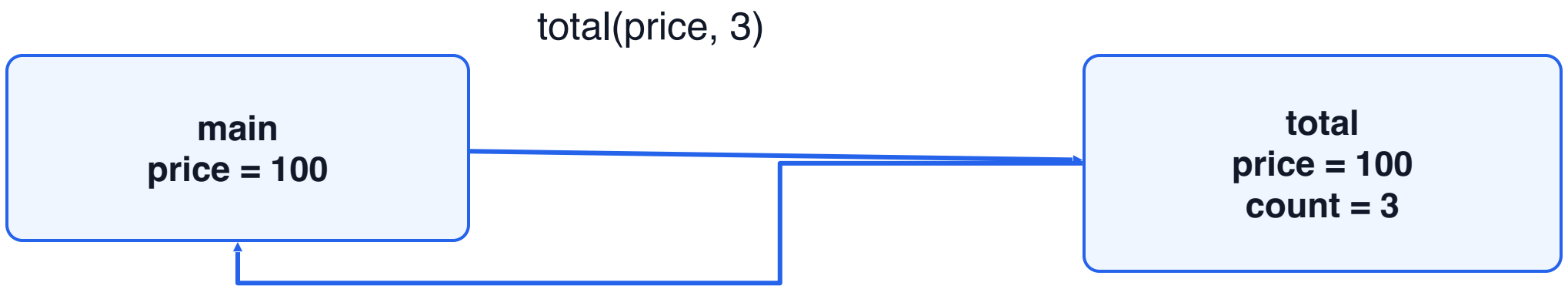
System.out.println(first);
System.out.println(second);
```

100 и 3 — аргументы первого вызова.

Каждый вызов имеет свои параметры и локальные переменные.

Метод возвращает 300 и 100 для двух разных вызовов.

Что происходит при вызове



Аргументы вычисляются, их значения копируются в параметры.

return и void

Два метода внутри класса Main

```
static boolean isPassed(int score) {
    return score >= 60;
}

static void printResult(int score) {
    if (!isPassed(score)) {
        System.out.println("Незачёт");
        return;
    }
    System.out.println("Зачёт");
}
```

return завершает текущий метод.

Метод boolean возвращает значение.

void выполняет действие без результата.

Вывести значение в консоль и вернуть значение — разные действия.

Область видимости

Фрагмент внутри main

```
int count = 2;

if (count > 0) {
    String message = "Есть задачи";
    System.out.println(message);
}

// System.out.println(message); // ошибка
```

message доступна
только внутри блока
её объявления.

count доступна
и внутри if,
и после него.

Локальные переменные не получают значение по умолчанию.

Java передаёт аргументы по значению

Содержимое класса Main

```
static void change(int value) {
    value = 100;
}

public static void main(String[] args) {
    int number = 5;
    change(number);
    System.out.println(number);
}
```

Параметр value получил копию 5.

Присваивание меняет только value.

number остаётся 5.

То же правило действует и для ссылок. Позже посмотрим пример.

Практика: выделяем метод

- Напишите static int countPassed(int[] scores)
- Верните число оценок не ниже 60
- Вызовите метод для двух разных массивов
- Печать результата оставьте в main

Метод должен работать и с пустым массивом.

Разбор: метод countPassed

Метод внутри класса Main

```
static int countPassed(int[] scores) {
    int passed = 0;
    for (int score : scores) {
        if (score >= 60) {
            passed++;
        }
    }
    return passed;
}
```

Массив приходит через параметр.

Результат выходит через return.

Счётчик принадлежит одному вызову.

countPassed(new int[]{59, 60, 90}) вернёт 2.

03. Классы и объекты

Собираем данные и действия одной задачи в одном типе

Зачем нужен собственный класс

- У задачи есть название и признак завершения
- Задачу можно завершить и проверить её статус
- Таких задач в программе может быть много
- Класс Task описывает их общую структуру и поведение

Объект — конкретная задача со своими значениями полей.

Первый класс Task

Отдельное объявление класса Task

```
class Task {
    String title;
    boolean done;

    void complete() {
        done = true;
    }
}
```

title и done — поля.
Хранят состояние.

complete — метод.
Меняет состояние
конкретной задачи.

Поля объявлены в классе, локальные переменные — внутри метода.

Конструктор задаёт начальное состояние

Класс Task с конструктором

```
class Task {
    String title;
    boolean done;

    Task(String initialTitle) {
        title = initialTitle;
        done = false;
    }
}
```

Имя конструктора совпадает с классом.

Тип результата не указывается, даже void.

Конструктор выполняется при создании объекта через new.

new создаёт объект

Фрагмент внутри main

```
Task first = new Task("Изучить Git");
Task second = new Task("Изучить Java");

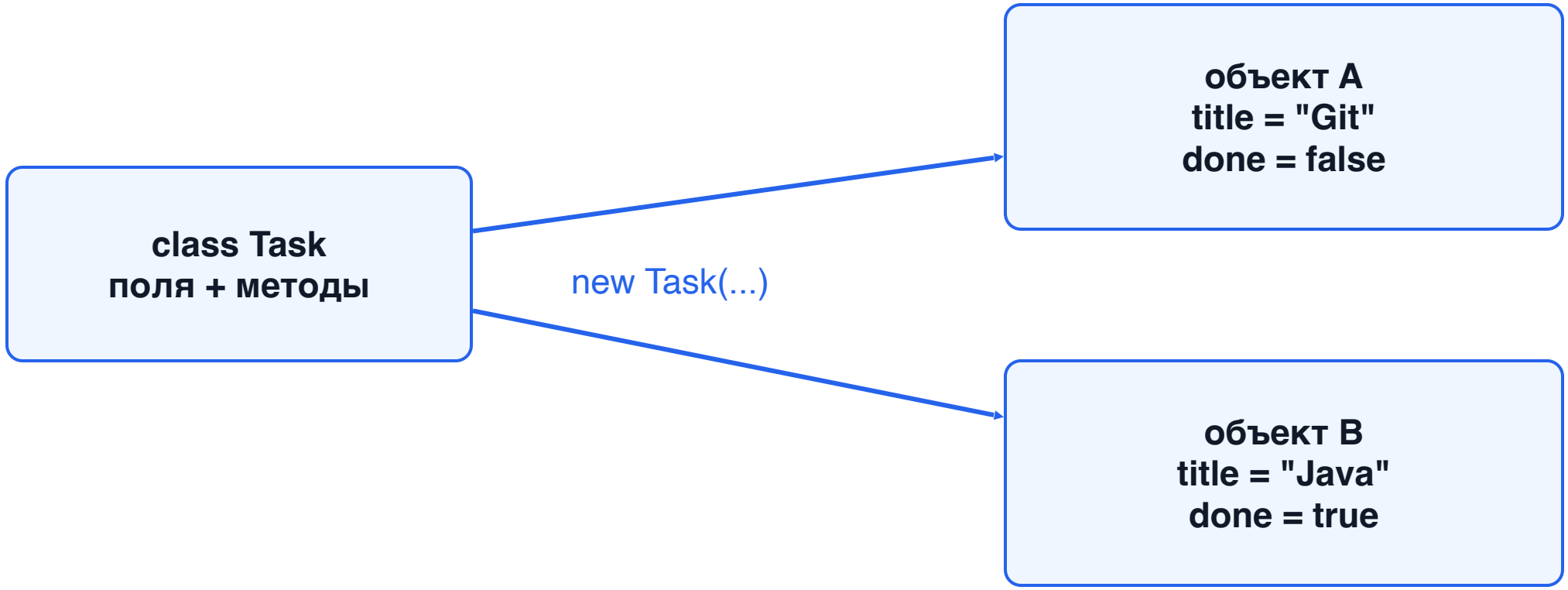
System.out.println(first.title);
System.out.println(second.title);
```

Task — тип переменной.

new Task(...) создаёт объект и вызывает конструктор.

first и second содержат ссылки на разные объекты.

Один класс, разные объекты



Поля экземпляра у каждого объекта свои.

this: текущий объект

Класс Task с одноимёнными полем и параметром

```
class Task {
    String title;

    Task(String title) {
        this.title = title;
    }
}
```

this.title — поле объекта.

title — параметр конструктора.

this помогает отличить поле от одноимённого параметра.

Вызов метода объекта

Фрагмент внутри main

```
Task task = new Task("Изучить Java");

System.out.println(task.done);
task.complete();
System.out.println(task.done);
```

Через точку выбираем поле или вызываем метод объекта.

В complete поле done относится к task.

Вывод: false, затем true.

Присваивание ссылки не копирует объект

Фрагмент внутри main

```
Task a = new Task("Git");
Task b = a;

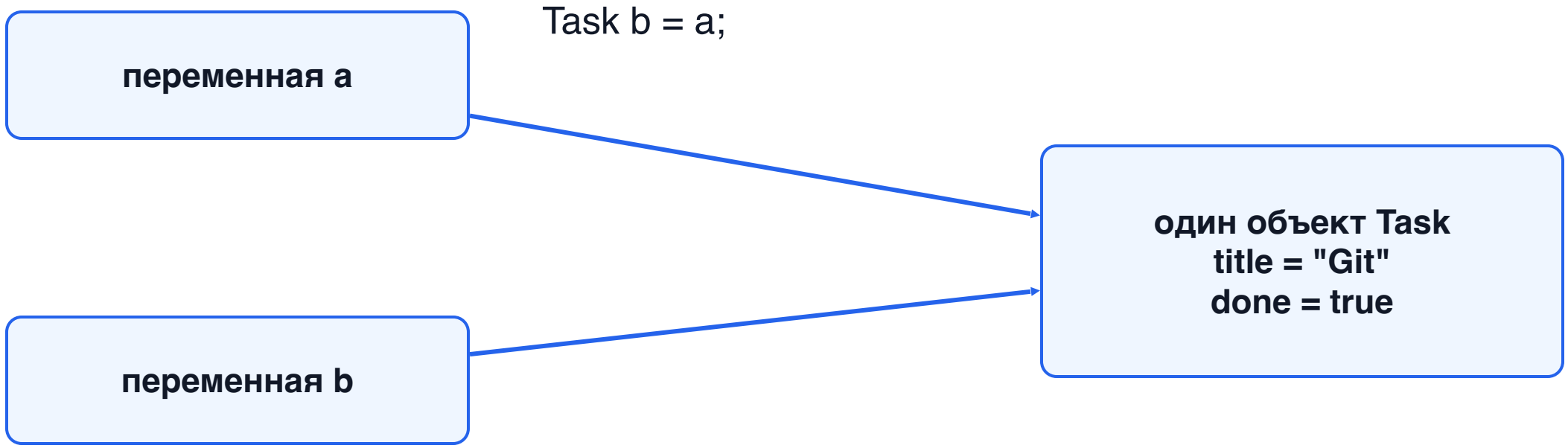
b.complete();
System.out.println(a.done);
System.out.println(a == b);
```

После b = a
две переменные
указывают на
один объект.

Изменение видно
через обе ссылки.

Вывод: true и true.

Две ссылки на одну задачу



b.complete() изменяет тот объект, на который ссылается и a.

null: объект отсутствует

Фрагмент внутри main

```
Task task = null;

if (task != null) {
    task.complete();
}

// task.complete(); // NullPointerException
```

null — отсутствие ссылки на объект.

У null нельзя вызвать метод или прочитать поле.

Проверяем отсутствие там, где оно допустимо по смыслу.

Короткое замыкание &&

Фрагмент внутри main

```
String title = null;

if (title != null && !title.isBlank()) {
    System.out.println(title);
} else {
    System.out.println("Нет названия");
}
```

Если слева false, правая часть && не вычисляется.

Поэтому isBlank() не вызывается у null.

Порядок частей условия здесь важен.

Инкапсуляция: состояние за методами

Полный учебный класс Task

```
class Task {
    private String title;
    private boolean done;

    Task(String title) { this.title = title; }
    void complete() { done = true; }
    boolean isDone() { return done; }
    String getTitle() { return title; }
}
```

private ограничивает доступ к полям.

Внешний код использует методы, которые разрешил автор класса.

Не обязательно создавать setter для каждого поля.

static и обычный метод

Полный класс Counter

```
class Counter {
    private int value;

    void increment() { value++; }
    int getValue() { return value; }

    static int twice(int number) {
        return number * 2;
    }
}
```

increment работает с конкретным счётчиком.

Counter.twice(3) не требует объекта Counter.

В static-методе нет this.

final запрещает новое присваивание

Фрагмент внутри main

```
final int limit = 10;
// limit = 20; // ошибка компиляции

final Task task = new Task("Git");
task.complete();

// task = new Task("Java"); // ошибка
```

final у переменной запрещает заменить её значение.

Объект по ссылке может оставаться изменяемым.

final-ссылка не делает весь объект неизменяемым.

Копия ссылки передаётся в метод

Содержимое Main; Task из примера с private-полями

```
static void change(Task task) {
    task.complete();
    task = new Task("Другая");
}

public static void main(String[] args) {
    Task original = new Task("Git");
    change(original);
    System.out.println(original.getTitle());
    System.out.println(original.isDone());
}
```

complete меняет общий объект.

Присваивание task меняет только локальный параметр.

Вывод: Git и true. Переменная original не переназначена.

Практика: какие задачи завершены?

Фрагмент внутри main

```
Task a = new Task("A");
Task b = a;
Task c = new Task("A");

b.complete();

System.out.println(a.isDone());
System.out.println(c.isDone());
System.out.println(a == c);
```

Нарисуйте переменные и объекты.

Сколько здесь вызовов new? Сколько объектов?

Одинаковое название ещё не означает один объект.

04. Организация и общие контракты

Пакеты, интерфейсы и основы полиморфизма

package и import

Отдельный файл TaskStore.java

```
package course.tasks;

import java.util.ArrayList;
import java.util.List;

public class TaskStore {
    private List<Task> tasks = new ArrayList<>();
}
```

package задаёт пространство имён.

import позволяет писать List вместо java.util.List.

В проекте путь обычно соответствует пакету: course/tasks/TaskStore.java.

Кому доступен член класса

Модификатор	Краткое правило
private	Внутри класса; для наших примеров это сам Task
Без модификатора	Коду того же пакета
protected	Коду того же пакета и наследникам с ограничениями
public	Коду, которому доступен сам тип

Для начала: поля private, наружу только нужные операции.

Интерфейс описывает доступные операции

Два объявления типов

```
interface Notifier {
    void send(String message);
}

class ConsoleNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println(message);
    }
}
```

Notifier задаёт контракт send.

ConsoleNotifier реализует этот контракт.

@Override помогает проверить намерение.

Код может зависеть от операции, а не от конкретной реализации.

Полиморфизм: тип и реализация

Фрагмент внутри main

```
Notifier notifier = new ConsoleNotifier();
notifier.send("Задача завершена");
```

Тип переменной: Notifier.

Класс объекта: ConsoleNotifier.

Выполняется метод реального объекта.

Другую реализацию можно подставить через тот же интерфейс.

Наследование: extends и super

Самостоятельный пример двух классов

```
class Task {
    private String title;
    Task(String title) { this.title = title; }
    String label() { return title; }
}

class UrgentTask extends Task {
    UrgentTask(String title) { super(title); }
    @Override
    String label() { return "! " + super.label(); }
}
```

UrgentTask — разновидность Task.

super(title) вызывает конструктор родителя.

label переопределён.

Класс наследует один класс, а реализовать может несколько интерфейсов.

Композиция и абстрактный класс

- Композиция: TaskStore содержит задачи в поле
- Наследование: UrgentTask является разновидностью Task
- Экземпляр abstract-класса нельзя создать напрямую
- Абстрактный метод задаёт операцию без реализации

Сначала выбираем понятную модель, потом механизм языка.

Перегрузка и переопределение

	Перегрузка / overload	Переопределение / override
Что меняется	Список параметров	Реализация у наследника
Пример	print(int) и print(String)	UrgentTask.label()
Когда выбирается	По типам аргументов при компиляции	По классу объекта во время выполнения

Только разного возвращаемого типа для перегрузки недостаточно.

== и equals: два разных вопроса

Фрагмент внутри main

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a == b);
System.out.println(a.equals(b));
```

== сравнивает ссылки.

String.equals сравнивает содержимое строк.

Вывод: false и true.

enum: ограниченный набор вариантов

Объявление enum и фрагмент main

```
enum Status {
    NEW, IN_PROGRESS, DONE
}

// В main:
Status status = Status.NEW;
if (status == Status.NEW) {
    System.out.println("Новая задача");
}
```

Status задаёт свой тип.

Можно использовать только объявленные значения.

Значения enum сравнивают через ==.

Опечатка вроде Status.DNOE обнаружится при компиляции.

record: компактный тип для данных

Объявление record и фрагмент main

```
record User(long id, String name) {}

// В main:
User user = new User(1L, "Анна");
System.out.println(user.name());
System.out.println(user);
```

Компилятор создаёт конструктор, методы id() и name().

Также доступны equals, hashCode и toString.

Компоненты record нельзя переназначать после создания.

05. Коллекции и ошибки

Храним много объектов и обрабатываем неудачные операции

Как хранить несколько значений

Тип	Идея	Пример задачи
Массив T[]	Фиксированная длина, доступ по индексу	Заранее известные 7 значений
List<T>	Порядок и повторяющиеся элементы	Список задач
Set<T>	Уникальные элементы	Набор тегов
Map<K, V>	Соответствие ключа значению	Найти задачу по идентификатору

Выбираем структуру по операции, которую хотим выполнить.

List и ArrayList

Фрагмент внутри main

```
List<String> titles = new ArrayList<>();
titles.add("Git");
titles.add("Java");
titles.add("Git");

System.out.println(titles.get(1));
System.out.println(titles.size());
titles.remove(0);
```

List — интерфейс.
ArrayList — реализация.

Индексы с нуля.
Повторы разрешены.
Размер меняется.

Импорты: java.util.List и java.util.ArrayList.

Параметр типа и классы-обёртки

Фрагмент внутри main

```
List<Integer> scores = new ArrayList<>();
scores.add(70);
scores.add(90);

int first = scores.get(0);
System.out.println(first);

// scores.add("пять"); // ошибка компиляции
```

<Integer> ограничивает тип элементов.

int автоматически упаковывается в Integer и обратно.

List<int> недопустим. Для чисел используем List<Integer>.

Список объектов Task

Фрагмент внутри main

```
List<Task> tasks = new ArrayList<>();
tasks.add(new Task("Git"));
tasks.add(new Task("Java"));
tasks.get(0).complete();

for (Task task : tasks) {
    if (!task.isDone()) {
        System.out.println(task.getTitle());
    }
}
```

Список хранит ссылки на задачи.

for-each даёт очередную задачу.

Выводим только незавершённые.

Этот код соединяет классы, условия и цикл.

Set: уникальные значения

Фрагмент внутри main

```
Set<String> tags = new HashSet<>();
tags.add("java");
tags.add("backend");
tags.add("java");

System.out.println(tags.size());
System.out.println(tags.contains("java"));
```

Повторное добавление "java" не создаёт второй элемент.

HashSet не обещает порядок обхода.

Импорты: java.util.Set и java.util.HashSet.

Map: значение по ключу

Фрагмент внутри main

```
Map<Long, String> names = new HashMap<>();
names.put(1L, "Анна");
names.put(2L, "Олег");
names.put(1L, "Аня");

System.out.println(names.get(1L));
System.out.println(names.get(9L));
```

Ключ 1L связан с одним значением.

Повторный put заменяет значение.

get отсутствующего ключа вернёт null.

Импорты: java.util.Map и java.util.HashMap.

List.of: список без изменения состава

Фрагмент внутри main

```
List<String> fixed = List.of("Git", "Java");
// fixed.add("SQL"); // UnsupportedOperationException

List<String> editable = new ArrayList<>(fixed);
editable.add("SQL");
System.out.println(editable.size());
```

List.of создаёт неизменяемый по составу список.

Для добавления создаём ArrayList.

Не все объекты одного интерфейса разрешают одинаковые изменения.

Три вида ошибок

Когда	Пример	Как искать
Компиляция	int count = "три";	Прочитать диагностику компилятора
Выполнение	Вызов метода у null	Прочитать исключение и стек
Неверный результат	Проверили > 60 вместо >= 60	Проверить примеры и границы

Успешная компиляция ещё не доказывает правильность программы.

Исключение прерывает обычный ход

Фрагмент внутри main

```
System.out.println("Начало");
int value = Integer.parseInt("abc");
System.out.println("Конец");
```

Строка "abc"
не является
целым числом.

Возникает
NumberFormatException.

«Конец» не выведется, если исключение никто не обработал.

try и catch

Фрагмент внутри main

```
String text = "abc";

try {
    int value = Integer.parseInt(text);
    System.out.println(value);
} catch (NumberFormatException error) {
    System.out.println("Введите целое число");
}
System.out.println("Продолжаем");
```

try содержит операцию, которая может не удаться.

catch обрабатывает конкретный тип исключения.

Не оставляем пустой catch и не скрываем причину ошибки.

throw: запрещаем некорректную задачу

Конструктор внутри Task с полем private String title

```
Task(String title) {
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException(
            "Название не должно быть пустым"
        );
    }
    this.title = title;
}
```

Конструктор проверяет вход.

При ошибке объект не возвращается вызывающему коду.

Если объект создан успешно, у него есть допустимое название.

throws и checked-исключения

Метод в Main; импорты перечислены в заметках

```
static String readText(Path path)
        throws IOException {
    return Files.readString(path);
}
```

throws сообщает: ошибка может уйти вызывающему коду.

IOException нужно обработать или объявить дальше.

throw бросает исключение, throws описывает его в объявлении.

Ресурсы: try-with-resources

Фрагмент внутри main

```
Path path = Path.of("tasks.txt");
try (BufferedReader reader =
         Files.newBufferedReader(path)) {
    System.out.println(reader.readLine());
} catch (IOException error) {
    System.out.println("Не удалось прочитать файл");
}
```

reader автоматически закроется при выходе из try.

Закрытие произойдёт и при исключении.

GC освобождает память. Файл нужно закрывать своевременно.

Как читать ошибку и пользоваться отладчиком

- Посмотреть тип исключения и сообщение
- Найти ближайшую строку своего кода в стеке вызовов
- Поставить breakpoint перед проблемным действием
- Проверить значения и пройти выполнение по шагам

Сначала воспроизводим проблему, затем меняем код.

06. Собираем приложение

Несколько задач, завершение и вывод результата

Практика: небольшой список задач

- Task хранит название и признак завершения
- Конструктор отвергает null и пустое название
- complete() завершает задачу; isDone() возвращает статус
- В main создайте список из трёх задач и завершите одну
- Выведите названия оставшихся задач и их количество

Проверьте пустой список, все завершённые и повторный complete().

Разбор: поля и конструктор Task

Файл Task.java, первая часть

```
class Task {
    private final String title;
    private boolean done;

    Task(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException(
                "Название не должно быть пустым");
        }
        this.title = title;
    }
    // Методы на следующем слайде
}
```

Название обязательно и не меняется.

done изначально равно false.

Состояние скрыто за private.

Сохраняем класс в Task.java.

Разбор: методы Task

Продолжение Task.java, внутри фигурных скобок класса

```
void complete() {
    done = true;
}

boolean isDone() {
    return done;
}

String getTitle() {
    return title;
}
```

Добавляем методы внутрь класса Task.

Повторный complete оставляет done равным true.

Методы выражают операции, которые нужны вызывающему коду.

Разбор: Main связывает всё вместе

Фрагмент внутри main

```
List<Task> tasks = new ArrayList<>();
tasks.add(new Task("Git"));
tasks.add(new Task("Java"));
tasks.add(new Task("SQL"));
tasks.get(0).complete();

int remaining = 0;
for (Task task : tasks) {
    if (!task.isDone()) {
        System.out.println(task.getTitle());
        remaining++;
    }
}
System.out.println("Осталось: " + remaining);
```

Импорты List и ArrayList.

Результат:
Java
SQL
Осталось: 2

Каждая часть программы отвечает на небольшой понятный вопрос.

Проверяем понимание

- Чем класс отличается от объекта?
- Что хранится в переменной ссылочного типа?
- Чем поле отличается от параметра и локальной переменной?
- Чем return отличается от println и break?
- Что изменится после Task b = a и b.complete()?

Ответьте своими словами и покажите короткий пример.

Что изучать следующим блоком

- Писать тесты для методов и классов
- Уверенно работать с коллекциями и равенством объектов
- Разбирать ошибки и организовывать проект
- После этого переходить к HTTP, JSON и backend-фреймворку

Для первого backend-проекта нужны понятные классы и методы.

Дополнительно: два термина

Можно пропустить при первом прохождении

var выводит тип, но не отменяет его

Фрагмент внутри main

```
var count = 3;
var title = "Java";

// count = "три"; // ошибка компиляции

System.out.println(count);
System.out.println(title);
```

Компилятор выводит int и String из начальных значений.

Тип переменной не меняется.

В начале обучения явный тип делает пример понятнее.

First-class citizens: о чём этот термин

- Так называют сущности, с которыми можно работать как со значениями
- Например: сохранять, передавать аргументом и возвращать
- В Java так работают со ссылками на объекты
- Ключевое слово class объявляет тип и относится к другой идее

Для базового кода важнее понимать класс, объект и ссылку.

Материалы для повторения

- Dev.java: Language Basics, Classes and Objects
- Dev.java: Interfaces, Collections Framework, Exceptions
- Helsinki Java MOOC: упражнения по основам и ООП
- Java API: точное описание метода и его исключений

Ссылки на конкретные темы находятся в заметках к слайдам.